

EAS Build & Déploiement Mobile

Du code au store — compiler, distribuer et publier

EAS Build	Android	iOS	Keystore
-----------	---------	-----	----------

TestFlight	Play Store	App Store
------------	------------	-----------

Parcours de formation — Développement d'Applications Mobiles

Djiguitech · Social Seller · Tous niveaux

■ Compétences visées

- ✓ Comprendre le cycle de vie complet d'une application mobile : dev → test → production
- ✓ Distinguer Expo Go, Development Build et Production Build
- ✓ Configurer eas.json avec les profils development, preview et production
- ✓ Compiler un APK Android et un IPA iOS via EAS Build
- ✓ Gérer les keystores Android et les certificats iOS
- ✓ Publier une application sur le Google Play Store et l'App Store

Table des matières

Chapitre 1 — Le cycle de vie d'une application mobile

- 1.1 Les environnements : dev, staging, production
- 1.2 Expo Go vs Development Build vs Production
- 1.3 Le rôle d'EAS dans l'écosystème Expo
- 1.4 Architecture d'un projet EAS

Chapitre 2 — Configuration EAS

- 2.1 Installation et authentification
- 2.2 eas.json — les trois profils
- 2.3 Variables d'environnement par profil
- 2.4 EAS Secrets pour les valeurs sensibles

Chapitre 3 — Build Android

- 3.1 Le processus de compilation Gradle
- 3.2 Keystore — signature et gestion
- 3.3 APK vs AAB
- 3.4 Installation et test sur device

Chapitre 4 — Build iOS

- 4.1 Prérequis Apple Developer
- 4.2 Certificates et provisioning profiles
- 4.3 Simulator vs device réel
- 4.4 TestFlight pour les testeurs

Chapitre 5 — Publication et mises à jour

- 5.1 Préparer le build de production
- 5.2 Soumettre sur le Play Store
- 5.3 Soumettre sur l'App Store

5.4 EAS Update — mises à jour OTA

Le cycle de vie d'une application mobile

1.1 Les environnements : dev, staging, production

Toute application professionnelle distingue plusieurs environnements. L'environnement de développement est celui du quotidien : rechargement à chaud, logs détaillés, base de données locale ou de test. L'environnement de staging reproduit la production le plus fidèlement possible : même données (anonymisées), mêmes services, version candidate pour les tests finaux. L'environnement de production sert les vrais utilisateurs : performances optimisées, logs minimalistes, monitoring actif.

En développement mobile, ces environnements se traduisent par différents builds de l'application. Le Development Build inclut des outils de debug (expo-dev-client, menu de développement, accès aux logs en direct). Le Preview Build est un build de production sans les outils de debug, distribué aux testeurs internes. Le Production Build est le build final signé, soumis aux stores.

1.2 Expo Go vs Development Build vs Production

Expo Go est une application téléchargeable gratuitement qui permet de charger rapidement votre code via un QR code. C'est l'outil idéal pour débiter et tester des fonctionnalités de base. Sa limite est importante : Expo Go ne peut utiliser que les modules inclus dans son propre bundle. Les modules natifs custom — expo-notifications, expo-secure-store avec configuration avancée, des bibliothèques de graphiques natifs — ne fonctionnent pas.

Le Development Build est un APK/IPA personnalisé de votre application qui inclut expo-dev-client. Il offre la même expérience de rechargement à chaud qu'Expo Go, mais avec l'intégralité de vos modules natifs. C'est le standard recommandé pour tout projet sérieux. Le Production Build est compilé avec des optimisations (minification, suppression du code de debug, tree-shaking) et signé avec votre keystore/certificat de distribution.

Comparaison des modes de lancement Expo

Critère	Expo Go	Development Build	Production Build
Installation	App Store / Play Store	APK direct ou EAS	App Store / Play Store
Modules custom	Non	Oui	Oui
Dev tools	Oui	Oui	Non
Performance	Normale	Normale	Optimisée
Distribution	Scan QR code	Lien EAS ou ADB	Store public
Usage	Prototypage	Développement quotidien	Utilisateurs finaux

1.3 Le rôle d'EAS dans l'écosystème Expo

EAS (Expo Application Services) est la plateforme cloud d'Expo pour la compilation, la distribution et la publication d'applications. EAS Build compile votre application dans le cloud — vous n'avez pas besoin d'installer Xcode (macOS uniquement) ni Android Studio. EAS Submit soumet automatiquement votre build aux stores. EAS Update pousse des mises à jour JavaScript directement sur les appareils des utilisateurs sans passer par le store (OTA — Over The Air).

Configuration EAS

2.1 Installation et authentification

```
# Installation globale du CLI EAS
npm install -g eas-cli

# Se connecter à son compte Expo
eas login

# Initialiser EAS dans le projet
eas init

# → Crée ou associe le projet sur expo.dev
# → Ajoute le projectId dans app.json

# Vérifier la configuration
eas build:inspect --profile development --platform android
```

2.2 eas.json — les trois profils

Le fichier eas.json à la racine du projet définit les profils de build. Chaque profil configure le type de distribution, les options de compilation, et les variables d'environnement. Les profils peuvent hériter d'autres profils via extends pour éviter la duplication.

```
// eas.json – configuration complète
{
  "cli": { "version": ">= 16.0.0" },
  "build": {
    "development": {
      "developmentClient": true, // Inclure expo-dev-client
      "distribution": "internal", // Pas pour les stores
      "ios": { "simulator": false },
      "env": {
        "EXPO_PUBLIC_API_URL": "https://...railway.app",
        "EXPO_PUBLIC_ENV": "development"
      }
    },
  },
},
```

```

"preview": {
  "distribution": "internal",
  "env": {
    "EXPO_PUBLIC_API_URL": "https://...railway.app",
    "EXPO_PUBLIC_ENV": "staging"
  }
},
"production": {
  "autoIncrement": true, // Incrémente le versionCode automatiquement
  "env": {
    "EXPO_PUBLIC_API_URL": "https://...railway.app",
    "EXPO_PUBLIC_ENV": "production"
  }
},
"submit": { "production": {} }
}

```

2.3 Variables d'environnement par profil

EAS distingue deux types de variables d'environnement. Les variables `EXPO_PUBLIC_*` sont embarquées dans le bundle JavaScript et accessibles dans le code de l'application. Les variables sans ce préfixe sont disponibles uniquement au moment de la compilation (pour les modules natifs). Les valeurs sensibles (clés API, secrets) ne doivent jamais être dans `eas.json` — elles doivent être stockées comme EAS Secrets.

```

# Créer un secret EAS (valeur chiffrée, jamais visible en clair)
eas secret:create --scope project --name EXPO_PUBLIC_SUPABASE_ANON_KEY \
--value 'eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...'

# Lister les secrets
eas secret:list

# Les secrets sont injectés automatiquement au moment du build
# Jamais exposés dans eas.json, les logs, ou le code source

# Dans le code – accès normal
const SUPABASE_URL = process.env.EXPO_PUBLIC_SUPABASE_URL;

```


Build Android

3.1 Le processus de compilation Gradle

La compilation Android passe par le système de build Gradle. EAS exécute ce processus dans le cloud sur des machines Linux. Les étapes sont : installation des dépendances npm, génération du code natif Android (expo prebuild), compilation avec Gradle (qui télécharge les dépendances Android Maven), et signature de l'APK/AAB avec le keystore. Ce processus prend typiquement 5 à 15 minutes.

```
# Lancer un build Android development
eas build --profile development --platform android

# Suivre le build en temps réel
# → EAS affiche l'URL du log et un lien de téléchargement à la fin

# Voir tous vos builds
eas build:list

# Télécharger le dernier build Android
eas build:download --platform android

# Build local (requiert Android Studio installé)
eas build --profile development --platform android --local
```

3.2 Keystore — signature et gestion

Le keystore Android est un fichier de certificat qui signe votre application. Google Play Store vérifie que chaque mise à jour est signée avec le même keystore — si vous perdez votre keystore, vous ne pouvez plus mettre à jour votre application sur le Play Store. Vous devriez publier une nouvelle application depuis zéro. EAS gère et stocke votre keystore de façon sécurisée, mais il est indispensable d'en faire un backup.

```
# EAS génère automatiquement un keystore si vous n'en avez pas
# (répondre Y à 'Generate a new Android Keystore?')

# Voir les credentials Android de votre projet
eas credentials --platform android

# Télécharger un backup du keystore
eas credentials --platform android
```

```
# → Sélectionner 'Download existing keystore'

# → Stocker en lieu sûr (gestionnaire de mots de passe, coffre-fort)

# Utiliser un keystore existant

eas credentials --platform android

# → Sélectionner 'Upload keystore'
```

■■ Attention

Faire un backup du keystore Android AVANT de changer de machine ou de compte EAS. Sans le keystore original, il est impossible de publier des mises à jour sur le Play Store pour une application existante. Ce backup doit être stocké dans un gestionnaire de mots de passe sécurisé, pas dans le dépôt Git.

3.3 APK vs AAB

APK (Android Package) est le format d'installation directe — c'est ce que vous installez manuellement sur un téléphone de test. AAB (Android App Bundle) est le format requis par le Google Play Store depuis 2021. Google optimise l'AAB pour chaque appareil : seul le code et les ressources nécessaires sont téléchargés par chaque utilisateur, réduisant la taille de l'installation. Pour la distribution interne (tests), utilisez APK. Pour le Play Store, utilisez AAB (EAS le génère automatiquement avec le profil production).

Build iOS

4.1 Prérequis Apple Developer

La compilation iOS requiert un compte Apple Developer (99 \$/an). Ce compte donne accès aux outils de signature, à TestFlight pour la distribution beta, et au processus de soumission sur l'App Store. Sans ce compte, il est impossible de compiler une app iOS pour un device réel (le simulateur ne nécessite pas de compte payant). EAS gère automatiquement la création et la rotation des certificats et provisioning profiles.

```
# Build iOS development (device réel)
eas build --profile development --platform ios

# EAS gère automatiquement :
# - Apple Distribution Certificate (certificat de signature)
# - Provisioning Profile (liste des devices autorisés)
# - Push Notification Key (pour les notifications)

# Enregistrer les devices des testeurs (pour les builds internes)
eas device:create

# → Génère une URL à ouvrir sur l'iPhone pour enregistrer le device
```

4.2 TestFlight pour les testeurs

TestFlight est la plateforme officielle d'Apple pour la distribution beta. Vous soumettez un build production sur TestFlight, ajoutez les emails des testeurs, et ils reçoivent une invitation à installer l'application depuis l'app TestFlight. C'est le chemin obligatoire pour distribuer des builds iOS à des personnes extérieures à votre équipe de développement.

Publication et mises à jour

5.1 Préparer le build de production

Avant de soumettre aux stores, plusieurs vérifications s'imposent. Les métadonnées de l'application (nom, description, screenshots, icônes) doivent être préparées. La version doit être correctement incrémentée. Le build doit être testé sur des appareils réels représentatifs. Les permissions (caméra, notifications, position) doivent être justifiées dans les métadonnées.

```
# Build de production
eas build --profile production --platform android
eas build --profile production --platform ios

# Soumettre automatiquement après le build
eas build --profile production --platform android --auto-submit

# Ou soumettre un build existant
eas submit --platform android
eas submit --platform ios
```

5.2 EAS Update — mises à jour OTA

EAS Update permet de pousser des mises à jour JavaScript directement sur les téléphones des utilisateurs sans repasser par le store. C'est possible parce que le code JavaScript est téléchargeable dynamiquement — seul le code natif compilé nécessite une mise à jour via le store. EAS Update est idéal pour corriger des bugs rapidement ou déployer des petites fonctionnalités sans attendre le processus de review des stores (qui peut prendre 24-72h).

```
# Installer expo-updates
npx expo install expo-updates

# Pousser une mise à jour OTA sur la branche production
eas update --branch production --message 'Fix: correction du calcul du total'

# Les utilisateurs reçoivent la mise à jour au prochain lancement de l'app
# (ou en arrière-plan selon la configuration)

# Voir l'historique des updates
eas update:list
```

■ À retenir

- ✓ Development Build = Expo Go + modules natifs custom + rechargement à chaud
- ✓ eas.json : development (dev client), preview (testeurs), production (stores)
- ✓ Les secrets sensibles dans EAS Secrets — jamais dans eas.json ni le code
- ✓ Keystore Android = identité de l'app sur le Play Store — backup obligatoire
- ✓ AAB pour le Play Store, APK pour la distribution interne
- ✓ EAS Update permet des hotfixes sans repasser par le store

— ■ Exercices

Exercice 1 — Premier Development Build

Compiler et installer un Development Build Android.

- Configurer eas.json avec les trois profils
- Lancer eas build --profile development --platform android
- Télécharger et installer l'APK sur un device Android
- Lancer npx expo start --dev-client et scanner le QR depuis l'app installée

Exercice 2 — Gestion des secrets

Migrer les variables d'environnement vers EAS Secrets.

- Identifier les variables EXPO_PUBLIC_* actuellement en dur
- Créer les secrets avec eas secret:create
- Supprimer les valeurs hardcodées de eas.json
- Vérifier que le build suivant fonctionne correctement

Exercice 3 — Preview pour les testeurs

Distribuer un build preview à un testeur externe.

- Compiler eas build --profile preview --platform android
- Récupérer le lien de téléchargement depuis expo.dev
- Envoyer le lien au testeur et l'aider à installer l'APK
- Collecter les retours et identifier un bug à corriger

Bibliographie & Ressources

Documentation officielle

- docs.expo.dev/build/introduction — EAS Build
- docs.expo.dev/eas-update/introduction — EAS Update
- docs.expo.dev/submit/introduction — EAS Submit

Guides des stores

- developer.android.com/distribute — guide Play Store
- developer.apple.com/app-store — guide App Store
- developer.apple.com/testflight — documentation TestFlight